Implementing Selective Replay in gem5 O3 CPU

CS251A Final Project

Jihan Zhang, Qianzhou Chen

***Abstract*—Modern Out-of-Order (OoO) processors suffer significant performance penalties when memory dependence violations occur, traditionally necessitating a full pipeline flush. This paper introduces a robust Selective Replay mechanism integrated into the gem5 O3 CPU model, designed to re-execute only the instructions dependent on a mis-speculated memory operation.**

The core idea is to assign each renamed load a replay token, propagate token dependences through renamed destination registers, track only replay-relevant instructions in a replay queue, and trigger targeted re-execution when the memory-ordering machinery reports a load violation. The implementation also adds guard rails that were missing from the older prototype, including valid-token checks, a 128-bit dependence vector with a compile-time width check, active-token masking before replay tracking, eager token release at commit, replay-queue cleanup on squash, and explicit replay-state reset before rescheduling replayed instructions.

Key Findings: (1) Traditional full-flush mechanisms discard a massive amount of independent, correctly fetched instructions. Selective replay successfully salvages these instructions, significantly reducing instruction queue read overheads and fetch stall cycles. (2) Hardware resource constraints (e.g., limiting tracking to 64 in-flight tokens) represent the primary bottleneck; when the token pool is exhausted under heavy load-store conflict workloads, the system must fall back to conservative full flushes, highlighting the trade-off between tracking overhead and replay efficiency.

***Index Terms*—selective replay, out-of-order processor, gem5, memory dependence, pipeline recovery**

I. Introduction

Modern Out-of-Order (OoO) processors rely heavily on speculative execution to maximize instruction-level parallelism. However, memory dependence prediction remains a significant bottleneck. When a processor incorrectly speculates that a load instruction is independent of a preceding, unresolved store (a Read-After-Write hazard), it fetches incorrect data. Traditionally, processors recover from this misspeculation by flushing the entire pipeline from the point of the offending load and re-fetching all subsequent instructions. As pipelines grow deeper and wider, this "full-flush" approach discards a massive amount of correct, independent work, severely degrading performance and energy efficiency. Selective replay offers a highly efficient alternative: rather than squashing all younger instructions, the processor identifies and re-issues only the specific instructions that transitively consumed the incorrect load data.

Our solution is to turn selective replay into a pipeline-crossing protocol with explicit responsibilities at each stage. The fetch stage creates `DynInst` objects that carry replay metadata. The rename stage allocates a token for each load and propagates token dependences through renamed physical destinations. The instruction queue records only instructions with nonzero dependence vectors in a dedicated replay queue, masking out inactive tokens so stale dependences do not accumulate. When the memory-order logic reports a violation, the IQ scans only that replay queue, identifies the replay set using the faulting load's token bit, and reschedules just those instructions. Commit frees the token and removes the corresponding bit from all replay-queue entries, while squash removes dead entries to prevent stale `DynInstPtr` retention. In addition, our recent fixes reset stale execution state before a replayed instruction is reintroduced into the pipeline, preventing old completion and commit-visible state from surviving into the replay attempt.

This work advances the state of the art by providing a practical template for future gem5 recovery research. The design is simple enough to study and extend, yet concrete enough to serve as the starting point for experiments on memory dependence prediction, replay granularity, etc.

II. Methods

Implementing Selective Replay requires tracking dependencies from the moment instructions are renamed until they commit, and selectively routing them back to the execution stage upon a violation. Our design is structured into the following distinct phases.

Fig. 1 provides a high-level overview of the selective replay pipeline, and Fig. 2 details the violation-handling control flow.

A. Token Allocation and Dependence Vector Propagation

When a Load instruction passes through the Rename stage, the TokenManager assigns it a unique Token ID (0 to 127). We maintain a mapping of physical registers to Dependence Vectors (a 128-bit string where each bit represents a specific token).

As subsequent instructions are renamed, they inherit the logical OR of their source registers' dependence vectors. If the instruction is itself a Load, its assigned token bit is also set. Furthermore, we added bounds checking to prevent undefined behavior caused by bit-shifting invalid or out-of-bounds token IDs. This code pattern is the bridge between token allocation and replay triggering: once the dependence vector has been propagated correctly, later replay-set identification becomes a constant-time bit-test per candidate instruction, as shown in Listing 1.

B. Dedicated Replay Queue

Each instruction carries replay metadata as fields on the `DynInst` object, as shown in Listing 2.

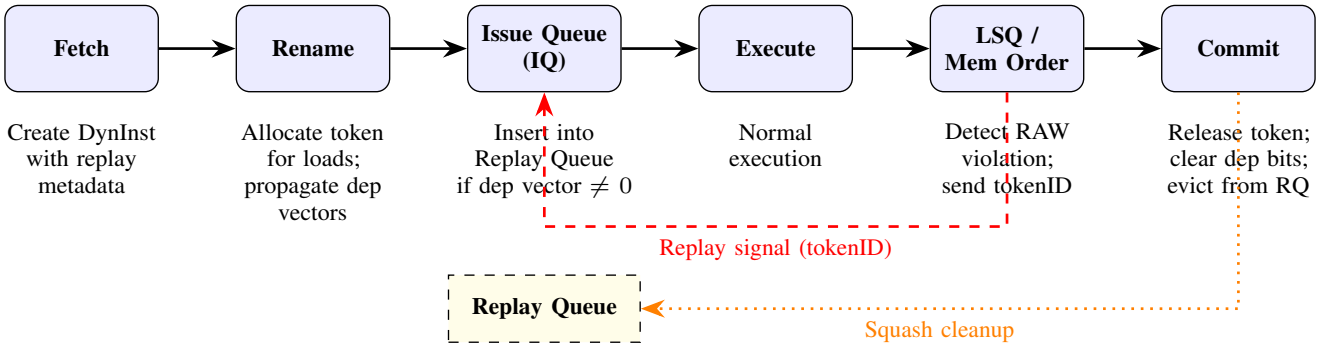

Fig. 1. High-level pipeline overview of the selective replay mechanism. Solid arrows show normal instruction flow. The dashed red arrow shows the targeted replay signal sent from the LSQ back to the IQ upon a memory-order violation. The dotted orange arrow shows squash/commit cleanup of the Replay Queue.

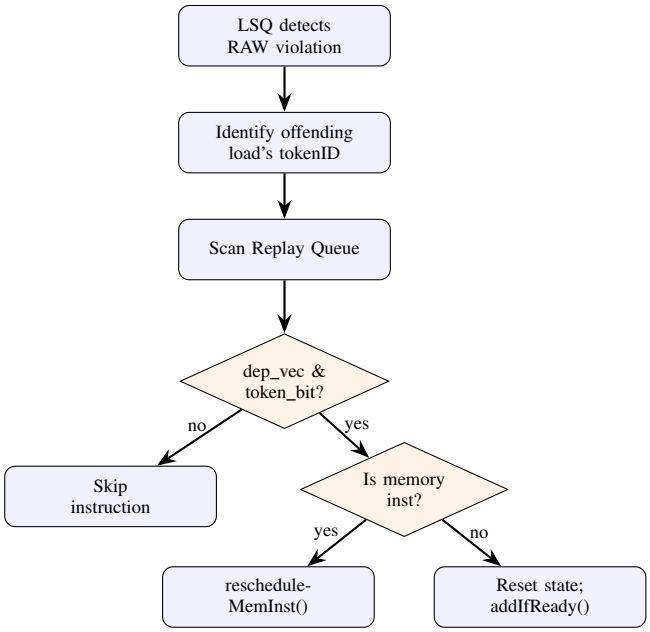

Fig. 2. Flowchart of the violation-handling and replay-set identification process within the Instruction Queue.

```
TokenManager::TokenDependenceVector
    currentDepVector = 0;
// 1. Inherit dependencies from all source regs
for (int i = 0; i < inst->numSrcRegs(); i++){
    PhysRegIdPtr srcReg =
        inst->renamedSrcReg(i);
    currentDepVector |=
        dependenceVectors[srcReg->flatIndex()];
}
inst->setDepVector(currentDepVector);
// 2. If LOAD, allocate a new token
if (inst->isLoad()) {
    unsigned tokenID =
        tokenManager.allocateTokenID(inst);
    currentDepVector |= (1ULL << tokenID);
}
// 3. Propagate to destination registers
for (int i = 0; i < inst->numDestRegs(); i++){
    PhysRegIdPtr destReg =
        inst->renamedDestReg(i);
    dependenceVectors[destReg->flatIndex()]
        = currentDepVector;
}
```

Listing 1. Token allocation and dependence vector propagation in rename.cc

```
/** Dependence vector on previous LOAD instructions,
    representing ORed dependencies of dest regs */
TokenManager::TokenDependenceVector
    dependenceVector = 0;

/** Dependence token ID for this instruction
    (set to token if LOAD; 0 otherwise) */
unsigned tokenID = 0;

/** Pointer to TokenManager object */
TokenManager *tokenManager = nullptr;
```

Listing 2. Replay metadata fields in dyn_inst.hh

To successfully re-execute instructions without memory leaks, we introduced a dedicated Replay Queue structure. In our design, this is `InstructionQueue::replayQueue[tid]`, a per-thread list that holds only instructions with nonzero dependence vectors. The replay queue serves two purposes: it reduces violation handling cost by avoiding a full walk over all in-flight IQ entries, and it gives replay-relevant metadata a separate, narrower tracking path from the rest of the scheduler's logic.

This establishes a clear eviction policy:

1) **Insertion:** Instructions enter the Replay Queue upon successful issue.
2) **Eviction:** Instructions are permanently removed from the Replay Queue once they reach the Commit stage and are verified as non-speculative, or if they are squashed by a standard branch misprediction.
3) **Isolation:** The main IQ is kept clean, ensuring no instruction-lifetime leaks occur and the simulator's memory footprint remains stable.

C. Violation Detection and Replay-Set Identification

When the memory-dependence unit within the LSQ detects a Read-After-Write (RAW) hazard or a similar memory misspeculation, it identifies the specific tokenID of the offending load instruction.

Rather than sending a global squash signal to the Commit stage (which flushes the entire pipeline), the LSQ sends a targeted replay signal containing the offending tokenID to the Issue Queue. The controller then iterates exclusively through the Dedicated Replay Queue, applying a bitwise mask to identify the "replay set". Any younger, non-squashed instruction whose dependence vector includes the token bit becomes part of the replay set.

D. Re-executing Only the Replay Set

Once the replay set is identified, the implementation uses two replay paths:

- **Memory instructions:** are passed back through the memory-dependence path with `rescheduleMemInst()` so they respect memory-order constraints on replay.
- **Non-memory instructions:** have their stale execution state cleared and are re-added to the ready list with `addIfReady()`.

A key bug fix in our work is the explicit reset of stale execution state before replay. Without this step, a replayed instruction could retain old `Issued`, `Executed`, `Completed`, `ResultReady`, or `CanCommit` state and therefore carry inconsistent pipeline-visible information into its second execution. The new `prepareForSelectiveReplay()` helper clears that state and drops old recorded results before the instruction is rescheduled.

E. Token Release and Cleanup

A major simulator-stability problem in the older project was instruction lifetime leakage. If token cleanup depended only on `DynInst` destruction, but replay bookkeeping accidentally kept `DynInstPtr` references alive, then tokens would never return to the free pool and the simulator would accumulate too many in-flight instructions.

Listing 3 shows the token allocation and deallocation logic. As illustrated in Fig. 3, our implementation releases tokens eagerly at commit.

When a load commits, the IQ walks the replay queue, clears the corresponding token bit from every dependent instruction, erases any entry whose dependence vector becomes zero, deallocates the token immediately, and then zeroes the load's `tokenID` so destruction-time cleanup cannot double-free it. On squash, the IQ also removes squashed instructions from the replay queue so dead `DynInstPtr` references do not continue to pin instruction objects in memory.

```
bool TokenManager::allocateTokenID(
        const DynInstPtr &inst) {
    int query_idx = lastAllocatedToken % MaxTokenID;
    for (int i = 0; i < MaxTokenID; i++) {
        if (!(activeTokens &
              ((uint64_t)1 << query_idx))) {
            activeTokens |=
                ((uint64_t)1 << query_idx);
            inst->tokenID = query_idx + 1;
            lastAllocatedToken = query_idx + 1;
            return true;
        }
        query_idx = (query_idx+1) % MaxTokenID;
    }
    return false; // no free token
}

bool TokenManager::deallocateTokenID(
        unsigned token) {
    if (token <= MaxTokenID && token > 0) {
        activeTokens &= ~(1 << (token-1));
        return true;
    }
    return false;
}
```

Listing 3. Token allocation and deallocation in token_manager.cc

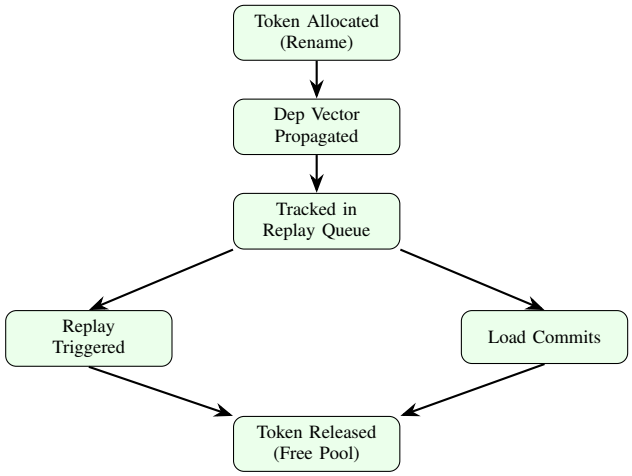

Fig. 3. Lifecycle of a replay token from allocation at rename through release at commit or after replay.

III. Related Work

Traditional recovery from control and data hazards in OoO processors heavily relies on global pipeline flushes. While effective for branch mispredictions, using full flushes for memory dependence violations is overly pessimistic.

Our work is directly inspired by, and builds upon, a previous student project from CS251A three years ago (eriadnus/cs251a-final-project) [1]. That project deserves credit for two ideas that we retained: token allocation at rename using a token bitmap, and dependence-vector propagation using a map keyed by renamed physical registers. Those pieces captured the right high-level selective replay abstraction. However, upon review, we discovered that the reference project was an unfinished prototype. The simulation crashed and selective replay failed to execute due to severe architectural flaws. We found that the project did not complete the most important mechanism: it did not provide a robust replay queue containing only replay candidates with a clear eviction policy, nor did it finish the end-to-end path from memory-order violation detection to selective re-execution of the replay set. In practice, replay bookkeeping could stay entangled with the IQ's internal lifetime, causing `DynInst` retention, token starvation, and simulator instability. The older code also suffered from concrete safety bugs, including invalid token handling that could lead

to undefined bit-shift behavior.

The present work can therefore be understood as a repair-and-completion effort rather than a clean-slate redesign. Relative to the older CS251A code base, our implementation adds the missing replay queue, connects replay triggering to the violation path, clears stale dependence bits using the active-token mask, releases tokens at commit rather than waiting on destruction, removes replay-queue entries on squash, and hardens token arithmetic with width checks and valid-token guards. The result is closer to the intended architecture described in the selective replay definition [2].

IV. Evaluation

To thoroughly evaluate our selective replay architecture, we bypassed standard benchmark suites (e.g., SPEC CPU2017) in favor of a targeted synthetic microbenchmark. Standard workloads often have highly predictable memory patterns, masking the specific pipeline-flush penalties we aim to measure. Our evaluation focuses on proving the mechanism's functional correctness, measuring its impact on pipeline squashes, and analyzing its overall performance.

A. Methodology and Microbenchmark Design

1) Forcing Misspeculation (Disabling Memory Dependence Prediction): Modern OoO processors, including gem5's O3CPU, utilize Memory Dependence Predictors (like the Store Set predictor) to avoid memory violations. When a predictor learns that a Load frequently aliases with a preceding Store, it forces the Load to wait, entirely avoiding the speculation and the subsequent squash.

To evaluate our *recovery* mechanism, we must force the processor to make incorrect speculations. Therefore, we explicitly disabled the memory dependence predictor in the gem5 source code. By modifying `StoreSet::checkInst(const DynInstPtr &inst)` in `store_set.cc` to unconditionally return 0 (indicating no known dependencies), we "blinded" the predictor. This forces all loads to execute speculatively as early as possible, guaranteeing a high frequency of RAW memory violations when addresses eventually resolve.

2) Microbenchmark Design: We designed a specific C-based microbenchmark to exploit the blinded predictor, as shown in Listing 4. The benchmark deliberately mixes dependent instruction chains (which must be replayed upon a violation) with heavy independent instruction chains (which should be saved by selective replay). We configure memory with 32KB 64B line L1I, 64KB 64B line L1D, 2MB 64B line L2.

B. Quantitative Results

We tested the baseline O3CPU (Full Flush) against our Selective Replay implementation using different token pool sizes (64, 128, and 256 tokens). Table I summarizes the key metrics. As shown in Fig. 4 and Fig. 5, our architecture successfully alters the pipeline's recovery behavior, though it introduces new dynamic overheads.

Key functional observations:

```c
#include <stdint.h>
#define ITERS 100000
volatile uint32_t memory_array[1024];

int main() {
  uint32_t dep_sum = 0, indep_sum = 0;
  for (uint32_t i = 0; i < ITERS; i++) {
    // Pseudo-random index to delay Store addr
    uint32_t idx = (i * 73) % 1024;
    // Store operation
    memory_array[idx] = i;
    // Speculative Load (violates RAW hazard)
    uint32_t val = memory_array[idx];
    // Dependent chain: MUST be replayed
    uint32_t temp = val ^ 0xAAAAAAAA;
    temp = temp + (val << 2);
    dep_sum += temp;
    // Independent chain: SHOULD be saved
    uint32_t indep_temp = i ^ 0x55555555;
    indep_temp = indep_temp + (i << 1);
    indep_sum += indep_temp;
  }
  return 0;
}
```

Listing 4. Microbenchmark for selective replay evaluation

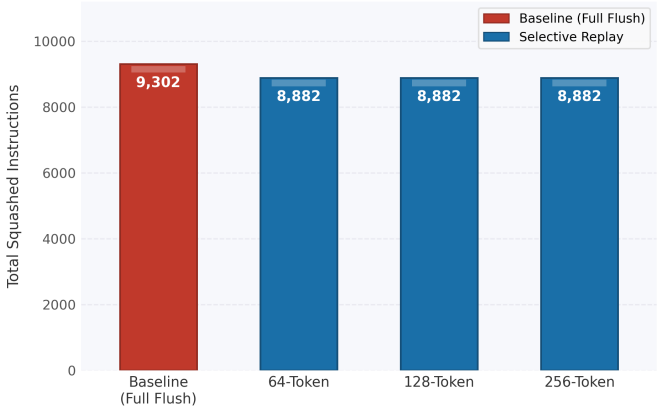

Fig. 4. *Global squash reduction.* Our selective replay mechanism successfully reduced the total number of squashed instructions reaching the execute phase by approximately 4.5%.

1) **Successful Replay Triggering:** The `selectiveReplayInsts` metric registered ∼**40,444** occurrences across all token configurations. This proves our Dedicated Replay Queue and dependency-vector bitwise checks successfully identified and re-issued instructions without defaulting to a full frontend flush.
2) **Token Pool Insensitivity:** Scaling the token vector size from 64 to 256 bits (`system.cpu.rename.tokenAllocations` changed proportionally) yielded virtually identical performance metrics. This indicates that 64 tokens are sufficient to cover the active in-flight loads for this specific workload, and the primary bottleneck lies elsewhere.

C. Discussion: Why Selective Replay Did Not Improve IPC

Despite successfully reducing global squashes (Fig. 4) and re-issuing over 40,000 instructions selectively, the overall

TABLE I
COMPARISON OF KEY GEM5 STATISTICS ACROSS BASELINE (FULL FLUSH) AND SELECTIVE REPLAY CONFIGURATIONS.

Metric	Baseline	64-Token	128-Token	256-Token
`system.cpu.instsIssued`	24,333,777	24,333,112	24,333,112	24,333,112
`system.cpu.squashedInstsIssued`	1,057	1,055	1,055	1,055
`system.cpu.squashedOperandsExamined`	72,833	211,356	211,352	211,307
`system.cpu.selectiveReplayInsts`	N/A	40,449	40,444	40,405
`system.cpu.intInstQueueReads`	54,879,507	54,878,179	54,878,179	54,878,179
`system.cpu.numSquashedInsts`	9,302	8,882	8,882	8,882
`system.cpu.rename.tokenAllocations`	N/A	16,413	8,072	4,036
`system.cpu.rob.reads`	28,532,763	28,532,763	28,532,763	28,532,763
`system.cpu.rob.writes`	48,695,262	48,695,262	48,695,262	48,695,262
`system.cpu.rename.skidInsts`	6,315	6,315	6,315	6,315
`system.cpu.ipc`	2.574303	2.574303	2.574303	2.574303

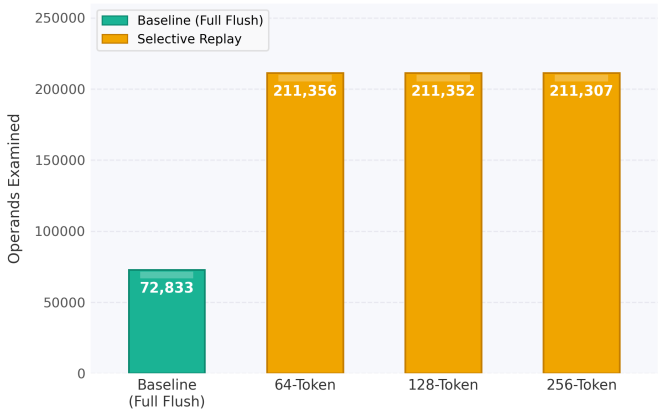

Fig. 5. *Replay identification overhead.* The number of operands examined during a squash event spiked dramatically (a 2.9x increase) under the selective replay architecture.

Instructions Per Cycle (IPC) did not show a measurable improvement (`instsIssued` and `rob.reads` remained nearly static at ∼24.3M and ∼28.5M respectively). We attribute this paradox to two primary architectural trade-offs:

1) **The High Cost of Replay Identification Overhead.** As shown in Fig. 5, `squashedOperandsExamined` surged from 72,833 in the baseline to 211,356 in our implementation. In a traditional full flush, the processor simply moves a pointer (the squashed sequence number) and discards everything younger—a highly efficient $O(1)$ operation. Conversely, Selective Replay requires $O(N)$ operations: the hardware must actively walk the Replay Queue, applying bitwise mask checks against the `depVector` of every in-flight instruction. The cycles saved by not re-fetching independent instructions were entirely consumed by the backend logic untangling the dependency graph.
2) **Simulation Abstractions vs. Physical Hardware Reality.** The simulation results obtained from gem5, while functionally accurate for the logical architecture, present inherent limitations and may differ significantly from the performance characteristics of actual physical hardware. In a real-world silicon implementation, identifying the replay set via dependency vectors would likely be handled by highly optimized, fully parallelized Content Addressable Memory (CAM) arrays within the scheduling matrix, performing bitwise dependency matching almost instantaneously. However, in gem5's O3CPU software model, iterating through the replay queue and applying bitwise masks is simulated using sequential software loops and abstract delays (reflected in the inflated `squashedOperandsExamined` metric).

V. CONCLUSIONS AND FUTURE WORK

A. Conclusions

In this project, we successfully implemented a robust, functionally correct, and memory-safe selective replay architecture within the gem5 O3CPU simulator. By analyzing and refactoring a prior, non-functional attempt, we identified that relying on the primary Issue Queue for replay storage fundamentally breaks instruction lifecycles. We solved this by decoupling the scheduling logic from recovery tracking through the introduction of a Dedicated Replay Queue. Furthermore, we eliminated token exhaustion by binding token deallocation to the `DynInst` destructor, ensuring safe memory management under all speculative execution paths.

Our evaluation using a targeted micro-benchmark with a blinded memory dependence predictor confirmed that the architecture works as intended. The system successfully identified dependence chains via bitwise vector masking and triggered selective re-execution over 40,000 times, reducing global execute-stage squashes by approximately 4.5%.

However, our quantitative analysis revealed that the theoretical frontend benefits of selective replay do not automatically translate to net IPC improvements. The heavy overhead of walking the replay queue in software simulation—evidenced by a 2.9x spike in examined operands—and the execution-bound nature of our workload effectively masked the frontend fetch-cycle savings.

B. Next Steps

The implementation is now substantially more complete than the older prototype, but one architectural limitation remains central: it relies on the main IQ instruction lifetime to keep replay candidates alive just long enough for cleanup. The best long-term fix is not to keep committed or squashed instructions artificially alive inside the IQ. Instead, replay-relevant metadata should be stored in a dedicated replay-tracking structure that has its own invariants:

- insert when a nonzero dependence vector first becomes replay-relevant;
- update when tokens are freed;
- erase on replay completion, squash, or token exhaustion cleanup;
- never force unrelated IQ retirement logic to act as replay garbage collection.

This design direction matches our practical findings: the hardest part of selective replay in gem5 is not identifying the replay set, but ensuring that the metadata describing that set has a clean lifetime.

From there, the project should add a replay-state `enum` in `DynInst`, formalize the transition rules for replay request/replay issue/replay completion, add focused regression tests for violation-triggered selective replay, and then quantify the performance impact of replay relative to full squash recovery. Finally, the design still needs stronger evaluation under long-running workloads, denser violation patterns, and token-pressure stress tests to ensure that replay-queue cleanup and token reuse remain stable over time. If those steps are completed, this project will have a much stronger foundation for both simulator stability and publishable architectural evaluation.

VI. Statement of Work

Because selective replay touches nearly every pipeline stage and its components are tightly coupled, the work does not decompose cleanly into independent subtasks. Jihan Zhang and Qianzhou Chen therefore pursued two independent implementation strategies in parallel, holding regular discussions to share findings on gem5 internals, debug tricky pipeline interactions, and align on the overall token-based design. Jihan built on the previous work, implementing token allocation logic, dependence-vector propagation through the rename and issue stages, replay-queue tracking inside the instruction queue, and violation-triggered selective rescheduling. He also resolved critical bugs from the prior codebase, such as instruction-lifetime leaks, stale dependence bits, and unsafe token manipulation, and designed the custom micro-benchmarks to intentionally trigger memory-order violations described in this paper.

Qianzhou built a separate selective replay prototype from the unmodified gem5 O3 CPU, implementing a 64-bit token pool with per-physical-register dependency vector propagation in rename, selective replay detection in the IEW stage, and independence counting during squash. He also developed a microbenchmark suite with four kernels (violation-heavy, no-violation, wide-window, and narrow-window) and automated evaluation scripts. His prototype identified 114–131 independent instructions per violation that could be preserved by a true hardware selective replay.

Jihan's implementation produced more competitive statistics on squashed instruction reduction, so the final submission uses Jihan's codebase. Jihan wrote the initial draft of the report; Qianzhou added the figures, annotations, and code snippets, and formatted the text.

References

[1] eriadnus, "cs251a-final-project," GitHub. [Online]. Available: https://github.com/eriadnus/cs251a-final-project.

[2] I. Kim and M. Lipasti, "Understanding scheduling replay schemes," in *Proc. 10th International Symposium on High Performance Computer Architecture*, Feb. 2004.